



Code review — variante-bac

Runde de review pe proiectul tău, cea mai nouă sus.

Un singur fișier, runda cea mai nouă sus. Rundele vechi rămân dedesubt.

Runda 3 — 2026-09-29 — sha dad5251

Un commit: `dad5251 finish tema+code review` . 13 fișiere atinse, 511 linii adăugate.

Ai atins **exact toate cele 7 fișiere cu**  **din runda 2** (3, 5, 8, 24, 25, 26, 52), plus `varianta20.h` și `varianta55.h` din lista  . Și ai rezolvat **integral fișa** `exercitii-doi-indici` — 434 de linii, toate cele 7 subprograme.

Codul NU a fost modificat — totul e doar aici.

Verificat prin **compilare și rulare**: fiecare header compilat separat cu `clang++ -std=c++17 -Wall -Wextra` , apoi rulat cu `-fsanitize=address,undefined` . Cele 7 subprograme din fișă au fost rulate pe **toate cele 20 de exemple din enunțuri**, nu doar pe datele pe care ți le-ai pus tu în `solutieN` .

Verdictul pe scurt.

Poarta de compilare: trecută. 32/32 headere. A treia rundă la rând. Warning-urile au scăzut de la 4 fișiere la 2.

Fișa a funcționat, și se poate demonstra. Exercițiul 5 din fișă era varianta 8 rescrisă cu alte cuvinte. Din 20 de exemple date în fișă, **19 ies corect**. Iar `interclasare` din `varianta8.h` — care la runda 2 pierdea elemente pe `a={2,4}` `b={1}` — dă acum răspunsul corect pe **toate cele 4** cazuri. Adică ai exersat mecanismul pe o cerință nouă și l-ai dus înapoi în fișierul vechi. E prima dată în trei runde când un fix migrează dintr-un loc în altul, în loc să rămână unde l-ai scris.

4 din 7  **închise curat**, 2 pe jumătate, 1 neatins. Plus M5 și M7 închise. Dintre cele 4 rezolvări „într-o linie” pe care ți le-am dat la finalul rundei 2, ai făcut 3.

Tiparul rundei 3 e nou și e cel mai instructiv de până acum: review-ul a devenit comentariu. În `varianta24.h`, liniile 5–6, ai copiat fix-ul meu în cod — ca **comentariu**:

```
///  
//!! CODE REVIEW "include <iomainip>"  
//! cout<<fixed<<setprecision(3)<<medie;
```

Linia 79 a rămas neschimbată. Programul afișează tot `4, 8 ,5.66667`. Ai transcris fix-ul, nu l-ai aplicat. Compară cu `varianta25.h`, unde ți-am dat același tip de indicație și ai **scris-o în cod** — și acolo constatarea e închisă.

Și tiparul vechi, sub altă formă. Fișa cerea **NU EXISTA** la trei exerciții. L-ai scris la `comune`  și la `perechi` , și l-ai uitat la `doarInA` . Aceeași regulă, trei locuri, două făcute. Compilatorul ți-a spus: `variable 'vf' set but not used`. Dar n-aveai cum să vezi warning-ul — vezi M1.

Starea constatărilor din runda 2

#	fișier	stare
B1 <code>varianta3.h</code> nu afișează nimic, <code>cif</code> neapelat	<code>varianta3.h</code>	🟡 PE JUMĂTATE — <code>cif</code> e apelat acum, <code>0</code> se afișează; palindromul nu se construiește → B3 mai jos
B2 <code>sub</code> gol + <code>interclasare</code> pierde elemente	<code>varianta8.h</code>	🟡 PE JUMĂTATE — <code>interclasare</code> ✅ CORECTĂ pe toate 4 cazurile ; <code>sub</code> nu există → B4 mai jos
B3 <code>4, 8, 5.66667</code> în loc de <code>5.667</code>	<code>varianta24.h</code>	🔴 NEATINS — fix-ul e scris ca comentariu → B1 mai jos
B4 mesajul <code>nu exista</code> numere prime	<code>varianta5.h</code>	✅ ÎNCHIS — <code>NU EXISTA</code> , și acum citește de la tastatură
B5 <code>f</code> schelet, <code>int k</code> în loc de <code>int &k</code>	<code>varianta25.h</code>	✅ ÎNCHIS — <code>void f(int n, int a[], int& k)</code> , dă <code>24</code>
B6 un <code>0</code> în plus la final	<code>varianta26.h</code>	✅ ÎNCHIS — <code>10 2 19 9 15</code>
B7 <code>i < d</code> → <code>k</code> cu 1 mai mic	<code>varianta52.h</code>	✅ ÎNCHIS — <code>i <= d</code> , dă <code>4</code>
M1 zero <code>fstream</code> în tot repo-ul	9 variante	🔴 NEATINS → M3 mai jos
M2 coliziuni de nume	tot repo-ul	🔴 aggravat (18× <code>solutie3</code>) → M5 mai jos
M3 <code>p</code> distruge tabloul apelantului	<code>varianta42.h</code>	🔴 NEATINS → M4 mai jos
M4 <code>k</code> nu se citește	<code>varianta46.h</code>	🔴 NEATINS → M7 mai jos
M5 <code>ordin(0)</code> dă 1	<code>varianta55.h</code>	✅ ÎNCHIS — <code>if (nr == 0) return 10;</code> · vezi nota de mai jos

#	fișier	stare
M6 prompturi la tastatură	6 locuri	🔴 NEATINS → M6 mai jos
M7 <code>nule</code> afișează	<code>varianta20.h</code>	✅ ÎNCHIS — afișarea a trecut în <code>solutie3</code>
C1 <code>shift</code> citește <code>x[n]</code>	<code>varianta23.h:31</code>	🔴 neatins
C3 variabile nefolosite	5 locuri	🟡 4 din 5 au dispărut odată cu rescrierile; rămâne <code>ct</code>
C2, C4–C9	—	🔴 neatins

O corectură de-a mea. La M5 runda 2 am scris că pe `n=1, a=(5), b=(0)` trebuia să iasă `50`. **Am greșit.** Definiția din enunț zice „`b[i]` concatenat cu `a[i]`, **altfel**”, iar `5 < 0` e fals — deci se ia ramura `altfel`, `0` concatenat cu `5`, adică `05 = 5`. Codul tău dădea răspunsul corect. Garda pe care ai pus-o e totuși corectă în sine (`0` are o cifră, deci ordinul lui e `10`) și am verificat că nu schimbă niciun rezultat: `ordin(0)` se atinge doar când `a` și `b` sunt amândouă `0`. Ai aplicat un fix bun pe o problemă care nu exista — verifică-mi constatările, nu le lua ca literă de lege.

Ce am rulat, ca să vezi dovada

Fișa `exercitii-doi-indici`, toate exemplele din enunțuri:

EX1 unire	a=(2,5,9) b=(1,5,7,12)	-> 1 2 5 5 7 9 12	p=7	corect
EX1 unire	a=(1,2,3) b=(7,8)	-> 1 2 3 7 8	p=5	corect
EX2 comune	a=(2,5,5,9,14) b=(1,5,9,20)	-> 5 9		corect
EX2 comune	a=(2,4) b=(1,3)	-> NU EXISTA		corect
EX2 comune	a=(3,3,3) b=(3,3)	-> 3		corect
EX3 doarInA	a=(2,5,5,9,14) b=(1,5,9,20)	-> 2 14		corect
EX3 doarInA	a=(4,6) b=(4,6,8)	-> (nimic) trebuia NU EXISTA		
EX3 doarInA	a=(1,2,3) b=(9)	-> 1 2 3		corect
EX4 reuniune	a=(2,5,5,9) b=(1,5,7,9)	-> 1 2 5 7 9	p=5	corect
EX4 reuniune	a=(4,4,4) b=(4,4)	-> 4	p=1	corect
EX5 alternant	a=(2,4,8,10,14) b=(3,5,11)	-> 2 3 4 5 8 11 14		corect
EX5 alternant	a=(2,4,6) b=(1)	-> 1 2		corect
EX5 alternant	a=(10,20) b=(1,3,5)	-> 1 10		corect
EX5 alternant	a=(2,4) b=(1,3)	-> 1 2 3 4		corect
EX6 perechi	a=(1,3,6,8,11) b=(2,4,5,9) s=10	-> 1 9/6 4/8 2		corect
EX6 perechi	a=(1,2) b=(3,4) s=100	-> NU EXISTA		corect
EX7 alKlea	a=(2,5,9) b=(1,5,7,12) k=4	-> 5		corect
EX7 alKlea		k=1 -> 1		corect
EX7 alKlea		k=7 -> 12		corect
EX7 alKlea	a=(1,2,3) b=(4,5) k=5	-> 5		corect

19 din 20. Variantele:

```

V3 : n=21523531 -> (nimic)      trebuia 53211235
V3 : n=12272351 -> 0            corect
V3 : cif(125854,5)=2  cif(10203040,0)=4      corect (punctul a)
V5 : 7 / 1125 2 314 101 37 225 12 -> 2 37 101      corect
V5 : 3 / 4 6 8                -> NU EXISTA      corect
V8 : a={2,4,8,10,14} b={3,5,11} -> 2 3 4 5 8 11 14      corect
V8 : a={2,4}   b={1}          -> 1 2            corect
V8 : a={2,4,6} b={1}          -> 1 2            corect
V8 : a={10,20} b={1,3,5}      -> 1 10          corect
V20 : {12,0,0,-3,-8,0}        -> 12 -3 -8 0 0 0      corect
V24 : n=5, {1,9,4,8,5}        -> "4, 8 ,5.66667" trebuia 5.667
V25 : a={5,4,3,2} n=4         -> 24              corect
V26 : exemplul din enunț      -> 10 2 19 9 15      corect
V52 : {16,17,8,31,8,2,10}     -> 4              corect
V55 : 3 / 5 12 7 / 0 3 100     -> 5 312 7100      corect
V55 : 1 / 5 / 0               -> 5              corect

```

Warning-uri, tot repo-ul (de la 4 fișiere la 2):

```

tema-exercitii-doi-indici.h:137: variable 'vf' set but not used
tema-exercitii-doi-indici.h:423: non-void function does not return
                                a value in all control paths
varianta24.h:61: unused variable 'ct'

```

Toate trei sunt bug-uri reale. Două sunt B2 și B5 de mai jos.

Critice

B1 — `varianta24.h` : fix-ul din runda 2 e scris ca comentariu

`varianta24.h:5-6` și `varianta24.h:79`

CUM E ACUM:

```
///CODE REVIEW "include <iomanip>"  
///cout<<fixed<<setprecision(3)<<medie;  
...  
cout << mini << ", " << maxi<<" , " << medie<<endl;
```

Rulat pe `n=5, x={1,9,4,8,5}` → `4, 8 ,5.66667` . Identic cu runda 2. Enunțul cere `5.667` .

DE CE: un comentariu nu e cod care așteaptă. Compilatorul șterge tot ce e după `//` înainte să înceapă să traducă ceva — linia ta nu ajunge niciodată la etapa în care ar putea produce un `cout` . Din punctul lui de vedere, fișierul e identic cu cel de la runda 2.

Iar `medie` e calculată **corect** — `5.66667` e chiar `17/3` , cu `(double) sum / n` scris cum trebuie. Deci algoritmul e gata de la runda 2. Ce lipsește e doar felul în care numărul ajunge pe ecran: `cout` afișează un `double` cu 6 cifre semnificative, fiindcă asta e precizia lui implicită. `setprecision(3)` singur ți-ar da `5.67` — tot 3 cifre **semnificative**. `fixed` schimbă înțelesul lui `setprecision` din „cifre semnificative” în „cifre **după virgulă**”. De-aia merg doar împreună, și de-aia trebuie amândouă.

Al doilea lucru: `mini` și `maxi` sunt pe linia de afișare din depanare, de când verificai că `p` întoarce valorile bune. Enunțul cere pe ecran **doar media**.

FIX:

```
#include <iomanip>  
...  
cout << fixed << setprecision(3) << medie;
```

(`<iomanip>` , nu `<iomanip>` — așa l-ai scris în comentariu.)

- același lucru de verificat la `varianta19.h`, singurul alt fișier care afișează un `double`.

B2 — `tema-exercitii-doi-indici.h`: `doarInA` nu afișează **NU EXISTA**

`tema-exercitii-doi-indici.h:137` și `:172`

CUM E ACUM: pe `a=(4,6)` și `b=(4,6,8)` nu se afișează nimic. Pe `a=(5,5)` și `b=(5,5)` la fel. Enunțul cere **NU EXISTA** în amândouă cazurile.

```
void doarInA(int a[], int n, int b[], int m) {
    bool vf = 0;
    while (i < n) {
        ...
        vf = 1;           // se pune
        ...
    }
    // si nimeni nu-l citește niciodată
}
```

DE CE: `vf` e scris în trei locuri și citit în zero. Un steag are sens doar în pereche: cine îl ridică și cine se uită la el la final. Aici jumătatea a doua lipsește — de-aia compilatorul a putut să-ți spună exact asta, cu cuvintele lui: `variable 'vf' set but not used`. E singurul caz în care compilatorul vede intenția, nu doar sintaxa: un `bool` pe care îl scrii și nu îl citești nu poate avea alt rost.

Lucrul care contează mai mult decât linia lipsă: în **același fișier**, `comune` (linia 117) și `perechi` (linia 368) au exact acest `if` și îl au corect. Trei exerciții cereau **NU EXISTA**, două îl au. Nu e lacună de cunoștințe — ai scris de două ori fix ce lipsește aici. De-aia „mai caut și celelalte locuri unde se aplică” e o mișcare separată de „repar locul semnalat”, și e cea care lipsește.

FIX:

```
if (vf == 0) {
    cout << "NU EXISTA";
}
}
```

B3 — `varianta3.h` : palindromul nu se construiește niciodată

`varianta3.h:70-73`

CUM E ACUM: pe `n = 21523531` nu se afișează nimic. Pe `n = 12272351` se afișează `0`, corect.

```
if (cif_impair > 0) {  
    cout << 0 << endl;  
}  
//incomplet  
}
```

DE CE: programul răspunde acum la o întrebare din două. Ai construit frecvențele și ai numărat câte cifre apar de un număr impar de ori — și asta rezolvă complet cazul „nu se poate”: la un număr cu 8 cifre, deci de lungime **pară**, un palindrom cere ca fiecare cifră să apară de un număr par de ori, fiindcă fiecare apariție din prima jumătate își are oglinda în a doua. Deci `cif_impair > 0` → `0`. Partea asta e corectă și e verificată.

Cazul complementar — `cif_impair == 0` — nu are niciun `else`. Și `cif_impair == 0` nu înseamnă „gata”, înseamnă „**se poate**, acum construiește-l”. Iar cel mai mare palindrom se obține punând în prima jumătate `fr[i]/2` copii din fiecare cifră, în ordine descrescătoare, și oglindindu-le. Pe `21523531` : fiecare din 1, 2, 3, 5 apare de 2 ori → jumătatea e `5321`, oglinda `1235`, răspunsul `53211235`.

Vestea bună: `cif` e **corect** și e **apelat** — `cif(125854, 5)` dă 2, `cif(10203040, 0)` dă 4. Punctul a) e luat, iar `fr[i] = cif(n, i)` e exact „apelul util” pe care îl cerea enunțul. Ce lipsește e ultimul pas, nu mecanismul.

FIX: un `else` pe `if`-ul de la linia 70, care parcurge `fr` de la 9 la 0 și scrie `fr[i]/2` copii ale cifrei `i`, apoi le oglindește.

B4 — `varianta8.h` : subprogramul `sub`, cerut de enunț, nu există

`varianta8.h:10` (enunțul) și `varianta8.h:54`

CUM E ACUM: algoritmul e corect — verificat pe toate 4 cazurile. Dar în fișier nu există niciun `sub`. Cuvântul apare o singură dată, în comentariul cu enunțul. La runda 2 `sub` era un schelet gol; acum a

dispărut cu totul.

```
// enuntul: "Subprogramul sub primește prin intermediul
//          parametrilor n si m ... a si b ...
//          Subprogramul va afisa pe ecran ..."

void interclasare(int a[], int dimA, int b[], int dimB,
                  int c[], int& dimC) // 6 parametri, nu afiseaza
void solutie4() // aici se afiseaza
```

DE CE: enunțul nu descrie doar un rezultat, descrie o **interfață**: numele `sub`, patru parametri (`n`, `m`, `a`, `b`) și — important — „**va afișa pe ecran**”. Adică afișarea face parte din ce trebuie să facă subprogramul, nu din ce face programul principal.

Al tău are șase parametri, dintre care doi (`c`, `dimC`) sunt un tablou ajutător pe care enunțul nu-l cere, și nu afișează. La corectare, subiectul III punctul b) se punctează pe „ai scris subprogramul cerut și l-ai folosit”, nu pe „iese numărul bun”. Un algoritm corect sub alt antet ia zero la b).

Și e util să vezi că nu ai nevoie de `c` deloc: dacă `sub` afișează pe loc, tot ce trebuie ținut minte între doi pași e **ultima valoare afișată** — un singur `int`. Exact ce ai făcut în `alternant` din fișă, care n-are niciun tablou rezultat și merge. Ai deja varianta bună scrisă, în celălalt fișier.

FIX: antetul `void sub(int n, int m, int a[], int b[])`, corpul mutat din `alternant` (fișă, exercițiul 5 — același mecanism, deja corect), `solutie4` rămâne doar cu datele și apelul.

B5 — `tema-exercitii-doi-indici.h`: `alKlea` iese din funcție fără `return`

`tema-exercitii-doi-indici.h:386` și `:423`

CUM E ACUM: pe `k` valid ($1 \leq k \leq n+m$) întoarce corect — toate 4 exemplele din fișă ies bune. Am forțat `k = 8` pe `n+m = 7` și programul a fost **oprit de sanitizer**:

```
runtime error: execution reached the end of a value-returning
function without returning a value
```

DE CE: `int alKlea(...)` promite un `int` pe fiecare drum posibil prin funcție. Tu ai patru `return`, toate în interiorul unor `if (ct == k)`. Dacă niciunul nu se declanșează, execuția ajunge la acolada de închidere a unei funcții care trebuia să întoarcă ceva — și atunci apelantul citește ce se nimerește în registrul de retur. Nu e „întoarce 0”: e comportament nedefinit, adică programul are voie să facă orice, inclusiv să pară că merge.

Compilerul îți spune asta direct: `non-void function does not return a value in all control paths`. În Visual Studio e warning-ul `C4715`, activ din start.

Asta e a **treia rundă la rând** cu aceeași regulă: runda 1 B7 (`varianta39.h`), runda 2 B5 (`varianta25.h`), acum aici. De fiecare dată ai închis fișierul semnalat. Regula e: **orice funcție care nu e void are nevoie de un return pe ultima linie a corpului**, nu doar în ramurile în care te-ai gândit că ajungi.

FIX: `return -1;` înainte de acolada de la linia 423.

🟡 Importante

M1 — proiectul tău compilează 1 fișier din 32

varianteBac.cpp · varianteBac.vcxproj

```
#include <iostream>
#include "varianta20.h"      // singurul header compilat
int main() { solutie3(); }
```

Proiectul are un singur `<ClCompile>` (`varianteBac.cpp`) și 31 de `<ClInclude>` . Un header apare în Solution Explorer, dar **nu e compilat** decât dacă un `.cpp` îl include. Deci când apeși F7, Visual Studio traduce `varianta20.h` și nimic altceva.

Consecința concretă: `tema-exercitii-doi-indici.h` — cele 434 de linii pe care le-ai scris acum — **nu e compilat de build-ul tău**. Cele două warning-uri din el (B2 și B5 mai sus) există, sunt gratuite, și n-au cum să ajungă la tine. Le-am văzut eu compilând fiecare header separat.

Ăsta e al doilea repo unde apare exact asta (la `vector-vectori` , runda 8, `main()` a plecat de pe `cautareBinara.h` și a lăsat 2 🟡 în urmă). Nu e o greșeală de cod, e o greșeală de **unealtă**: te muți pe fișierul următor și cel anterior iese din raza compilerului.

Cel mai simplu, până rezolvi M5: după ce termini un fișier, schimbă `#include` în `varianteBac.cpp` pe el și apasă F7 **înainte** de commit. Ia 5 secunde și îți dă toate warning-urile pe el.

M2 — reuniune afișează din interior și nu resetează p

tema-exercitii-doi-indici.h:185,238-243

Două lucruri, ambele în contrast cu cod scris de tine la 150 de linii distanță:

1. **Afișează din interior**. Enunțul cere să construiești `c` și să furnizeze `p` . Afișarea e treaba apelantului. Exact asta ai **corectat** în `varianta20.h` în același commit: ai mutat bucla de afișare din `nule` în `solutie3` . Aici ai făcut invers.
2. **Nu resetează p** . `unire` (linia 30) începe cu `p = 0` . `reuniune` nu. Am chemat-o de două ori cu același `p` : prima dată `p=5` , a doua `p=10` , cu rezultatul dublat în `c` . Merge azi doar fiindcă

`solutie4` pune `p = 0` înainte de apel — adică funcția depinde de o pregătire pe care nimeni n-a promis-o.

Fix: `p = 0;` la început, bucla de `cout` mutată în `solutie4`.

M3 — zero `fstream` în tot repo-ul (a treia rundă)

`grep -rn fstream *.h` nu întoarce nimic. Șase enunțuri cer fișier: `bac.in` (v39, v40), `bac.txt` (v46), `NUMERE.IN` (v26), `date.in` (v51, v52). La toate, datele sunt scrise în cod.

`varianta26.h` e acum **corectă algoritmic** — dar rezolvă exemplul din enunț, nu cerința. Începe cu `varianta39.h`, e cea mai scurtă:

```
#include <fstream>

ifstream fin("bac.in");

fin >> n;

for (int i = 0; i < n; i++) fin >> v[i];

fin.close();
```

M4 — `varianta42.h`: `p` distruge tabloul apelantului (neatins)

`varianta42.h:20–31`. `p` sortează descrescător **chiar tabloul primit**, prin `int a[]`, care e adresa tabloului din `solutie3`. După apel, datele apelantului sunt în altă ordine. Azi nu se vede fiindcă nimeni nu se mai uită la `a` după aceea. Enunțul nu cere modificarea lui `a`.

Același lucru la `varianta8.h:102–103`: `sortare(a, dimA)` și `sortare(b, dimB)`, pe două tablouri despre care enunțul spune explicit că sunt deja ordonate crescător.

M5 — 18 funcții cu numele `solutie3` (agravat)

```
18  solutie3      6  solutie4      3  p      2  sort
 2  solutie4b    2  solutie3b    2  solutie  2  f
```

De-aia trebuie să editezi `varianteBac.cpp` ca să rulezi altă variantă, și de-aia ai M1. Fix mecanic, o dată pentru tot repo-ul:

```
namespace v52 {  
    // tot conținutul fișierului  
}
```

Apoi `v52::solutie3()`. După asta poți include toate headerele în același `main` și rula tot cu un F5.

M6 — prompturi la tastatură (a treia rundă)

`varianta1.h:68,71` · `varianta6.h:20` · `varianta11.h:21` · `varianta12.h:54,57`. Un evaluator automat compară ieșirea caracter cu caracter; `n=` în față o face diferită de răspunsul așteptat.

M7 — `varianta46.h: k` nu se citește (neatins)

M8 — `int n;` necitit vs `int n = 0;`

`varianta55.h:33`. Rulat cu intrare goală, `cin >> n` **nu atinge** `n`, iar sanitizerul a raportat `index 106 out of bounds for type 'int[105]'`. Motivul e precis: `cin >> n` verifică mai întâi dacă are de unde citi (`sentry`); dacă dă direct de capătul intrării, partea care convertește cifrele nu e chemată deloc — și numai ea ar fi pus `0` în `n`. Deci `n` rămâne cu ce era în memorie, iar bucla `i <= n` merge oriunde.

În `varianta5.h:38` ai scris `int n = 0;`. Aceeași variabilă, două obiceiuri. La bac intrarea există mereu, deci nu te costă puncte — dar `= 0` la declarare costă 4 caractere.

Cleanups

- **C1** — `varianta23.h:31` : `for (i = 0; i < n; i++)` citește `x[n]` , în afara zonei permutate. Corect: `i < n - 1` . (C1 runda 2, neatins)
 - **C2** — `varianta24.h:61` : `int ct = 1;` declarat și nefolosit. Singurul rămas din C3 runda 2 — celelalte 4 au dispărut odată cu rescrierile.
 - **C3** — `varianta3.h:34` : `int cif = a % 10;` are **același nume** ca funcția `cif` în care se află. Merge (variabila locală acoperă funcția), dar în interiorul funcției nu mai poți scrie `cif(...)` . `c` sau `ultima` încheie subiectul.
 - **C4** — marcaje rămase pe fișiere care **acum sunt corecte**: `varianta20.h:6` (`//de revizuit`) și `varianta8.h:6` (`//de revazut`) — amândouă verificate corecte în runda asta. Șterge marcajul când închizi fișierul, altfel nu mai știi unde e lucrul nefăcut. Rămân reale: `varianta3.h:6,73` și `varianta24.h:8` .
 - **C5** — `comune` afișează `NU EXISTA` cu `endl` , `perechi` fără. Alege una.
 - **C6** — spațiu în plus la final de linie (C7 runda 2), `<cmath>` inclus în toate cele 32 de fișiere și nefolosit nicăieri, `Header.h` orfan (C5 runda 2), `int f(long a)` în `varianta5.h:27` (C4 runda 2), `count` peste `std::count` în `varianta18.h:18` (C2 runda 2), `a[i] % k` cu `k = 0` în `varianta16.h:19` , `fr[cif]++` fără gardă în `varianta14.h:22` . Toate neatinsse.
-

Before / After

#	fișier	acum	trebuie
B1	varianta24.h:79	<pre>cout << mini << ", " << maxi<<" , " << medie</pre>	<pre>cout << fixed << setprecision(3) << medie; + #include <iomanip></pre>
B2	tema...doi- indici.h:172	<pre>}</pre>	<pre>if (vf == 0) { cout << "NU EXISTA"; } }</pre>
B3	varianta3.h:72	<pre>} urmat de //incomplet</pre>	<pre>} else { /* construiește palindromul din fr */ }</pre>
B4	varianta8.h:54	<pre>void interclasare(int a[], int dimA, int b[], int dimB, int c[], int& dimC)</pre>	<pre>void sub(int n, int m, int a[], int b[]), care afișează</pre>
B5	tema...doi- indici.h:423	<pre>}</pre>	<pre>return -1; }</pre>
M2	tema...doi- indici.h:185	<pre>p folosit fără reset, cout în funcție</pre>	<pre>p = 0; la început, cout în soluție4</pre>
M8	varianta55.h:33	<pre>int n;</pre>	<pre>int n = 0;</pre>

✓ Ce e corect — verificat prin rulare

Fișa **exercitii-doi-indici** : 6 subprograme din 7 corecte pe toate exemplele. `unire` , `comune` , `reuniune` , `alternant` , `perechi` , `alKlea` — toate ies bune pe fiecare exemplu din enunț, inclusiv pe cazurile de la capete (`m = 1` , toate elementele lui `a` mai mici decât ale lui `b` , aceeași valoare repetată în ambele tablouri). `doarInA` greșește doar mesajul, algoritmul e corect.

Și, mai important decât numărul: **constrângerea de o singură parcurgere e respectată în toate 7**. Niciun `for` imbricat, niciun al treilea tablou la `alKlea` . Aia era toată miza fișei. La exercițiul 6 ai chiar și varianta cu bucle imbricate păstrată deasupra, comentată — ai scris-o, ai văzut că nu respectă cerința, ai rescris-o cu doi indici din capete opuse. Asta e exact mișcarea pe care voiam.

Trei lucruri merită spuse pe nume:

alternant — cele două contoare care nu avansează la fel. E cel mai greu din fișă și l-ai făcut cu un singur `int ultim` ca memorie. Când ultimul afișat e par, cauți în `b` primul impar strict mai mare; cât timp nu e, avansezi `j` **fără să afișezi**. Distincția între „am consumat un element” și „am afișat un element” e fix ce lipsea în `interclasare` la runda 2. Acum e clară în amândouă locurile.

comune și reuniune — săritul peste dubluri. `while (i < n && a[i] == valoare) i++;` pune amândouă convențiile de `i` („pe element” și „după element”) în același loc, o dată, controlat. La runda 2 asta era exact cauza lui B2: convențiile amestecate. Aici le-ai separat.

varianta20.h — afișarea mutată din nule în solutie3 . E M7 din runda 2, închis, și e singurul loc din commit unde ai schimbat un **design**, nu o linie. Un subprogram care „rearanjează și furnizează tabloul prin parametru” nu are ce să afișeze. Pentru asta vezi M2 — în `reuniune` ai făcut invers, în același commit.

Variante corecte, reverificate: v1(b), v6, v7, v11, v12, v14, v16, v17, v18, v19, v20, v23, v25, v26, v39, v40, v41, v42, v43, v46, v51, v52, v55, v62, v64, v66, plus `varianta8` pe algoritm și `varianta5` integral.

Q&A — răspunde-mi la astea trei

1. `varianta24.h` . Ai scris fix-ul pe linia 6, ca `//! cout<<fixed<< setprecision(3)<<medie;` , și programul afișează în continuare `5.66667` . **Fără să rulezi nimic:** în ce moment din traducerea fișierului dispăre linia aia, și de ce nu poate compilatorul să te avertizeze că ai scris un fix pe care nu l-ai aplicat? (A doua parte e cea care mă interesează.)
2. Fișa cerea `NU EXISTA` la trei exerciții. L-ai scris la `comune` , l-ai scris la `perechi` , l-ai uitat la `doarInA` . **Ce ai putea face înainte de commit** — o singură mișcare, nu trei — care ți-ar fi arătat că lipsește al treilea? (Indiciu: compilatorul ți-a spus deja, iar motivul pentru care nu l-ai auzit e M1.)
3. `alternant` din fișă și `interclasare` din `varianta8.h` rezolvă aceeași problemă și amândouă merg acum. Dar una are un al treilea tablou (`c`) și cealaltă nu. **Ce ține minte alternant între doi pași consecutivi**, și de ce e de ajuns atât? Răspunsul la asta e și fix-ul pentru B4.

Ce ai de făcut, în ordine

Într-o linie, patru constatări închise:

1. `tema-exercitii-doi-indici.h:172` — `if (vf == 0) cout << "NU EXISTA";` (B2)
2. `tema-exercitii-doi-indici.h:423` — `return -1;` (B5)
3. `varianta24.h` — `#include <iomanip>` , apoi linia 79 devine `cout << fixed << setprecision(3) << medie;` (B1)
4. `varianta23.h:31` — `i < n - 1` (C1, a doua rundă)

Apoi, în ordinea asta:

5. `varianta8.h` — B4: mută corpul lui `alternant` în `void sub(int n, int m, int a[], int b[])` . Codul e scris, e corect, trebuie doar antetul din enunț.
6. `tema-exercitii-doi-indici.h` — M2: `p = 0;` în `reuniune` , afișarea mutată în `solutie4` .
7. `varianta3.h` — B3: `else` -ul care construiește palindromul.

O dată pentru tot repo-ul, și îți simplifică tot restul:

8. M5 — `namespace vNN { }` în fiecare header. După asta `main()` poate include tot, iar M1 dispăre de la sine: un F7 îți dă warning-urile din toate cele 32 de fișiere, nu din unul.

9. M3 — citirea din fișier, începând cu `varianta39.h` .

M4, M6, M7, M8 și C2–C6 rămân pentru runda 5 — niciunul nu te blochează.

Runda 2 — 2026-09-23 — sha 02640ce

Patru commit-uri (a9c3abc , d068314 , 7bbcd5c , 02640ce). 16 variante noi, 5 atinse. Poarta de compilare: trecută, 30/30 headere, warning-uri în 4 fișiere din care 3 bug-uri reale.

Verdict: algoritmii s-au făcut buni — 13 din 16 variante noi dau răspunsul corect. Tiparul runde: *rezolvă altă problemă decât cea cerută* — subprogramul cerut ocolit, datele nu vin din sursa indicată, formatul de ieșire ignorat. Vechiul tipar („te oprești la un pas de final”) s-a concentrat în trei `if`-uri cu corpul gol.

#	constatare	fișier	stare la runda 3
B1	nu afișează nimic; <code>cif</code> definit și apelat de 0 ori	<code>varianta3.h</code>	🟡 pe jumătate — <code>cif</code> apelat ✅, palindromul lipsă → B3 runda 3
B2	<code>sub</code> tot gol; <code>interclasare</code> pierde elemente pe <code>a={2,4} b={1}</code>	<code>varianta8.h</code>	🟡 pe jumătate — algorithm ✅ corect pe 4/4 , <code>sub</code> inexistent → B4 runda 3
B3	<code>4, 8, 5.66667</code> în loc de <code>5.667</code>	<code>varianta24.h</code>	🔴 neatins — fix-ul scris ca comentariu → B1 runda 3
B4	mesajul <code>nu exista</code> <code>numere prime</code>	<code>varianta5.h</code>	✅ închis — NU EXISTA + citire de la tastatură
B5	<code>f</code> schelet: <code>if</code> gol, <code>int k</code> nu <code>int &k</code> , <code>int</code> fără <code>return</code>	<code>varianta25.h</code>	✅ închis — dă <code>24</code>
B6	un <code>0</code> în plus la final; trei convenții de indexare	<code>varianta26.h</code>	✅ închis — <code>10 2</code> <code>19 9 15</code>
B7	<code>i < d</code> în loc de <code>i</code> <code><= d</code>	<code>varianta52.h</code>	✅ închis — dă <code>4</code>
M1	9 variante nu citesc din sursa cerută; zero <code>fstream</code>	6 enunțuri	🔴 neatins → M3 runda 3
M2	coliziuni de nume (17× <code>solutie3</code>)	tot repo-ul	🔴 agravat (18×) → M5 runda 3
M3	<code>p</code> sortează in-place tabloul apelantului	<code>varianta42.h</code>	🔴 neatins → M4 runda 3

#	constatare	fișier	stare la runda 3
M4	<code>k</code> nu se citește, tabloul nu se construiește	<code>varianta46.h</code>	🔴 neatins → M7 runda 3
M5	<code>ordin(0)</code> dă 1	<code>varianta55.h</code>	✅ închis — garda pusă · ⚠️ exemplul meu (<code>5</code> → <code>50</code>) era greșit, vezi runda 3
M6	prompturi la tastatură	6 locuri	🔴 neatins → M6 runda 3
M7	<code>nule</code> afișează, deși n-ar trebui	<code>varianta20.h</code>	✅ închis — afișarea mutată în <code>solutie3</code>
C1	<code>shift</code> citește <code>x[n]</code> , în afara zonei permutate	<code>varianta23.h:31</code>	🔴 neatins → C1 runda 3
C2	<code>count</code> peste <code>std::count</code>	<code>varianta18.h:18</code>	🔴 neatins
C3	variabile declarate și nefolosite, 5 locuri	v24, v3, v25, v8	🟡 4 din 5 dispărute cu rescrierile; rămâne <code>ct</code>
C4	<code>int f(long a)</code> — îngustare tăcută	<code>varianta5.h:27</code>	🔴 neatins
C5	<code><cmath></code> nefolosit în toate fișierele; <code>Header.h</code> orfan	tot repo-ul	🔴 neatins
C6	marcaje <code>///??</code> rămase	5 fișiere	🟡 2 din 5 sunt acum pe fișiere corecte → C4 runda 3
C7	spațiu în plus la final de linie	9 fișiere	🔴 neatins

#	constatare	fișier	stare la runda 3
C8	stergere citește x[dim]	varianta24.h:49	● neatins
C9	a[i] % k cu k = 0; fr[cif]++ fără gardă	v16, v14	● neatins

Trei ● închise curat, două pe jumătate. Cele închise sunt exact cele unde primise linia de cod; cele pe jumătate sunt acolo unde constatarea avea două lucruri în ea.

Corecte la runda 2, spuse pe nume: inversarea prin apeluri repetate de `shift` (`varianta23`), Euclid în lanț (`varianta66`), `ordin()` izolat ca subprogram (`varianta55`), două contoare cu ritmuri diferite în `fibo` (`varianta62`).

Livrate în aceeași rundă: `exercitii-runda-2.pdf` (8 exerciții, câte unul per mecanism greșit) și `exercitii-doi-indici.pdf` (7 exerciții pe mecanismul din `varianta8.h`, cu constrângerea „o singură parcurgere”). Pe a doua a rezolvat-o integral — vezi runda 3.

Runda 1 — 2026-09-15 — sha 46a53db

Commit 46a53db tema (variantele 3, 5, 24, 39) plus verificarea integrală a celor din 4281854 1st push . Poarta de compilare: trecută, 14/14 headere, 4 warning-uri din care 3 bug-uri reale.

Verdict: din cele 4 fișiere ale temei, unul singur (varianta39) producea răspunsul cerut. Tiparul rundeii: *te oprești la un pas de final* — algoritmul scris, linia care îl leagă de răspuns lipsă.

#	constatare	fișier	stare la runda 2
B1	parametrii de ieșire transmiși prin copie, nu prin referință	<code>varianta24.h:18</code>	✅ închis
B2	<code>for (int i = i; ...)</code> — bucla pornește de la o valoare nedefinită	<code>varianta24.h:23</code>	✅ închis
B3	nu afișează nimic; media s-ar calcula în întregi	<code>varianta24.h:57</code>	🟡 pe jumătate → B3 runda 2
B4	cifra 9 nu e numărată niciodată (<code>i < 9</code> pe un tablou de 10)	<code>varianta3.h:48,62</code>	⬜ dispărut odată cu rescrierea
B5	palindrom: <code>0</code> nu se afișează, zeroul lipsește, mijlocul lipsește	<code>varianta3.h:50–66</code>	🔴 înrăutățit → B1 runda 2
B6	punctul b) nu e scris: <code>f</code> nu e apelat niciodată; mesaj greșit	<code>varianta5.h:44</code>	🟡 pe jumătate → B4 runda 2
B7	funcție <code>int</code> care nu întoarce nimic	<code>varianta39.h:15</code>	✅ închis
B8	<code>sub</code> e schelet: interclasează și nu afișează nimic	<code>varianta8.h:18–33</code>	🔴 neatins → B2 runda 2
M1	două variante nu pot exista în același proiect (nume duplicat)	tot repo-ul	🔴 agravat → M2 runda 2

#	constatare	fișier	stare la runda 2
M2	<code>sort</code> definit de două ori și suprapus peste <code>std::sort</code>	<code>varianta1.h:44</code> , <code>varianta12.h:28</code>	● neatins
M3	prompturile de la tastatură strică evaluarea automată	5 locuri	● neatins → M6 runda 2
M4	<code>varianta39</code> cere citire din <code>bac.in</code> , nu din <code>cin</code>	<code>varianta39.h:21</code>	● neatins → M1 runda 2
M5	subprogramul <code>nule</code> afișează, deși n-ar trebui	<code>varianta20.h:29–31</code>	● neatins → M7 runda 2
C1	<code>a[i] % k</code> cu <code>k = 0</code>	<code>varianta16.h:19</code>	● neatins
C2	<code>fr[cif]++</code> fără gardă pe interval	<code>varianta14.h:22</code>	● neatins
C3	<code>int f(long a)</code> — îngustare tăcută pe <code>return a</code>	<code>varianta5.h:32</code>	● neatins → C4 runda 2
C4	<code><cmath></code> nefolosit peste tot; <code>Header.h</code> orfan	tot repo-ul	● neatins → C5 runda 2
C5	marcaje <code>/// rămase</code>	5 fișiere	● neatins → C6 runda 2

Corecte la runda 1, verificate prin rulare: `varianta1.h` (b), `varianta7.h` , `varianta12.h` , `varianta16.h` , `varianta17.h` , `varianta20.h` , `varianta39.h` , plus `varianta6.h` , `varianta11.h` , `varianta14.h` fără obiecții de fond.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro